SOAT — Arquitetura da solução (Oficina API)

Participante

Tiago de Andrade Guapyassu Machado

Links da entrega

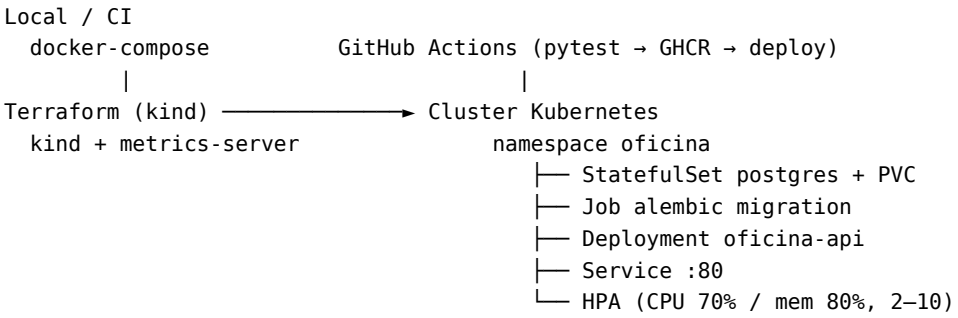
Repositório GitHub: https://github.com/TGuapyassu/pos_arquitetura

Vídeo demonstrativo (até 15 min): https://youtu.be/vIpxsSsmQwo

Aplicação em produção: https://pos-arquitetura.tguapyassu.com/

Desenho da arquitetura

Monólito FastAPI em camadas (DDD / hexagonal) implantado em Kubernetes, com PostgreSQL interno, migrations via Job, HPA e provisionamento local via Terraform (kind).



Recursos escolhidos

Camada	Tecnologia / recurso
API	Python 3.11+, FastAPI, Pydantic v2, JWT
Persistência	PostgreSQL 16, SQLAlchemy 2, Alembic
Arquitetura app	DDD + Clean/Hexagonal (ports & adapters)
Container	Docker multi-stage + docker-compose (dev)
Cluster local	kind (kindest/node:v1.29.2), cluster oficina
IaC	Terraform ≥1.5 (kind-cluster + k8s-platform)
Kubernetes	Namespace, StatefulSet, PVC, Job, Deployment, Service
Escalabilidade	HPA min 2 / max 10 — CPU 70%, memória 80%
Métricas	metrics-server (Helm via Terraform)
CI/CD	GitHub Actions → pytest → GHCR → k8s-deploy.sh
Registry	ghcr.io/<owner>/oficina-api:<sha>

Justificativa dos recursos

A aplicação é um monólito em camadas com regras de negócio no domínio e persistência relacional: o PostgreSQL garante ACID, FKs e schema versionado com Alembic. Docker e kind permitem reproduzir o ambiente localmente; o Terraform formaliza o cluster, o Postgres interno e o metrics-server. No Kubernetes, o Deployment da API, o Job de migrations e o HPA cobrem deploy idempotente e elasticidade sob carga. O GitHub Actions fecha o ciclo com testes, publicação da imagem no GHCR e deploy automatizado.